A Decentralized Collaborative Code Editor with Automated Distributed Version Control

1st Rajesh Phursule
Department of Information Technology
Pimpri Chinchwad College of Engineering
Pune, India
rajesh.phursule@pccoepune.org

2nd Sphurti Deshmukh
Department of Information Technology
Pimpri Chinchwad College of Engineering
Pune, India
sphurti.deshmukh@pccoepune.org

3rd Sumit Solanke
Department of Information Technology
Pimpri Chinchwad College of Engineering
Pune, India
sumit.solanke22@pccoepune.org

4th Sahil Shendkar
Department of Information Technology
Pimpri Chinchwad College of Engineering
Pune, India
sahil.shendkar22@pccoepune.org

5th Nikhil Shinde
Department of Information Technology
Pimpri Chinchwad College of Engineering
Pune, India
nikhil.shinde22@pccoepune.org

6th Aniket Polkar
Department of Information Technology
Pimpri Chinchwad College of Engineering
Pune, India
aniket.polkar22@pccoepune.org

***Abstract*—Collaborative software development usually depends on centralized code repositories and web-based services. Although widely adopted, such systems introduce security risks and availability limitations due to the dependency on continuous connectivity through organizational networks or VPNs and on central servers. A failure, attack, or misconfiguration at a single point may interrupt the entire development workflow and expose sensitive source code. This paper presents a decentralized collaborative code editor that eliminates the need for a central repository server while automating common version control operations. In the proposed system, each developer maintains a local copy of the codebase and collaborates directly with peers, while routine tasks such as saving changes, generating commits, and synchronizing updates are automatically handled in the background, decreasing manual intervention and human error. By distributing collaboration and version-control responsibilities across participating nodes, the system reduces the number of centralized points of vulnerability and supports continued development during network interruptions. The proposed methodology improves resilience, enables secure collaboration, and delivers a workable alternative to conventional centralized code-editing platforms by combining decentralized collaboration with automated version-control mechanisms.**

***Index Terms*—Collaborative Code Editing, Decentralized Systems, Distributed Version Control, Git Automation, Security, Peer-to-Peer Collaboration**

I. Introduction

In the current world, software development is no longer a solo job. Most software have been built as a team and these teams' members can come from various corners of the world. This means that, for instance, one member can be based in India while another is working from the USA and yet another person could be based in some other country. The main thing that comes into play in such a scenario is that these people should use some kind of technology like code editing tools and version controls [8], [11] among others.

Although collaborative working in the centralized mode presents a number of benefits such an approach also poses certain disadvantages. The tools used in collaborative programming use centralized repositories that store the code and facilitate coordination, secured with Virtual Private Networks (VPNs) [10]. Thus, the process of authentication, code synchronization, and history management have to go through certain central gateways that are built according to the client-server model.

With the rise in importance and size of software systems, failures and attacks of critical elements have become more severe. Problems such as outages, disruptions due to network failure or even attacks on sensitive data [2], [9] can disrupt the development and make the process difficult. For this reason, we think there needs to be an alternative that facilitates collaboration, but lessens dependency on a centralized solution. From our standpoint, doing this will lead to greater reliability in the process of software development.

Taking all the above into account, we suggest a decentralized collaborative code editor using a peer-to-peer approach with each member operating as an individual peer that has a private local repository. In this approach, version controlling is done automatically, and peer synchronization takes place with the direct exchange of updated codes between the peers with no involvement from a centralized server. The system we suggested has the capability of performing an offline-first workflow in which commiting changes happens locally. From our implementation, we found this to improve the reliability of the process.

II. Related Work

Collaborative editor is another method that can be found within team software development. The examples of such

software are GitHub and GitLab, which provide functionality such as online code editors, central repositories, ticketing system, and permission control [6], [8], [11]. Having a common repository helps to better organize cooperation between developers. Furthermore, there is Visual Studio Code Live Share, where developers can edit the same code at once [4]. We have also tried something similar for our project, and although they facilitate change management, they still use centralized architecture.

Even with all these capabilities, the vast majority of such applications remain based on a central system. For instance, during our experience with services like GitHub or GitLab, we noticed that actions like push, pull, and merge are carried out via their servers [6], [8], [11].

As a consequence, should there be any delays or disruptions in server performance, development would be affected. Moreover, even though applications like VS Code Live Share offer collaboration, they too need a reliable internet connection and rely on a central system to manage sessions [4]. Also, some corporate settings would require a virtual private network (VPN), adding another layer of dependence on gateway networks [7], [10].

From our own experience, such a dependence has some issues. For instance, should there be any problem with the internet connection or the central system, then the development will not proceed for some time. Moreover, having all data in one place may put us at higher risks in case of security breach [2], [7], [9].

However, once we used Git, we noted that it functions differently from the centralized system. Instead of storing the source codes in one repository, each individual keeps a local copy of the code and its history [6], [8], [11]. This made it possible for us to edit files, generate branches, and commit changes without any Internet connection [6]. Git guarantees data integrity through hashing [6].

In addition, while using Git, we observed that source code sharing among peers takes place via push, pull, and fetch commands [6], [11]. This made decentralization possible; but in real-life scenarios, most teams opt for GitHub or GitLab platforms [6], [8], [11].

In the course of our collaboration, merging and conflict resolution was often not an easy task. Conflicts could emerge when a number of people edited the same file, thus making it necessary to resolve them manually.

Having considered the research papers related to our project, we were able to note that quite a few authors regard peer-to-peer architectures as alternatives to centralized systems. These architectures involve several nodes with no central server that would coordinate the process, thus allowing them to survive the failure of one of the nodes [2], [9]. Furthermore, there are several technologies such as Operational Transformation and CRDT aimed at managing simultaneous edits by many users [3], [5], [11], [13]. There are such technologies as CodeFlow allowing real-time collaboration, however, they are not entirely decentralized due to their central management [12]. Another important feature of such tools is their focus on collaborative document editing.

Another factor is with regard to security. With centralized approach, all components such as source codes, history, credentials, etc. are kept in a single place [7]–[9]. Hence there is a higher likelihood of attack against the system. Also in case of virtual private network based architecture, most of the information relies on limited infrastructure which might prove to be a challenge [7], [10]. While going through the entire process, we realized that even centralized management of tokens and SSH keys could create issues if exposed.

As a result of the above factors, it was felt that there should be another way which would involve less reliance on the centralized system. This led us to choose a peer-to-peer approach in our work.

III. Methodology

This part gives an overview of the architectural approach used when creating the decentralized collaborative editor with automatic version control features. Instead of going into implementation specifics or providing experimental results, the emphasis is put on system architecture as well as the ways developers interact in the process of collaboration and how automation comes into play. The goal of this approach is to create a realistic framework for decentralized collaboration that could help solve problems related to centralized development systems.

A. Design Approach

The proposed solution takes a design-based approach for minimizing dependence on centralized collaboration spaces. Classic repository servers tend to create single points of failure and operational dependencies that limit availability [1], [2]. In this regard, a decentralized design model is adopted where a centralized repository is no longer necessary, but collaborative development and versioning support are maintained.

As illustrated by Fig. 1, ownership of the repository is distributed among autonomous peer nodes with no central coordination. Every user keeps an entire copy of their own repository and can communicate with other peers in a decentralized fashion. Under the peer-based approach, all peers function as equals, manage their respective development environment and version history autonomously, following the spirit of distributed systems [2], [3].

Distributed ownership, decentralized management and offline operation ensure fault tolerance, repository independence and continued development activity even when connected. Automation is another feature being used to minimize manual intervention and facilitate collaboration with less frequent use of version-control tools.

B. Decentralization Strategy

The strategy of decentralization is the foundation of both collaboration and versioning in the suggested model. The individual node is the holder of the local repository and directly interacts with other nodes to share updates, thereby

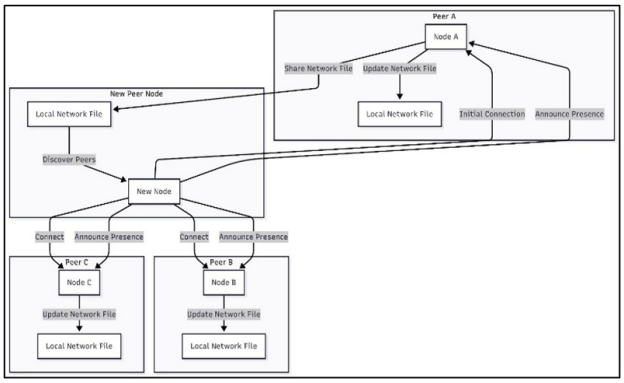

Fig. 1. Decentralized collaboration architecture showing autonomous peer nodes with no central coordination.

eliminating any centralized mechanism. Each node is seen as a complete repository with its own versioning system, hence it is capable of functioning independently [3], [5].

This strategy will enable collaboration to continue in the case of network outages, and there is no fallback strategy that can be considered. No consideration is given to hybrid approaches as the focus of the study is completely on decentralized behavior. Authenticated participants are assumed, but the degree of trust allowed is defined by the protocol only [9].

C. Model for Collaboration

Instead of employing real-time collaborative editing, the asynchronous peer-to-peer model is chosen. The updating of repositories is done by participants separately, meaning that the work can go on without any regard to time coordination. Repositories may temporarily differ and eventually unify after being synchronized.

Dissociation of conflict detection from synchronization preserves integrity of repositories and makes it possible to resolve conflicts manually when required [3], [5], [9], [13].

D. Automation Philosophy

Automation is applied to minimize human factor and simplify boring version control related actions. Actions like staging, committing, fetching, and synchronization are automated while retaining all required features of distributed version control [6], [8].

The fewer actions the user has to do, the easier and more efficient the collaboration is going to be, without compromising repository correctness, traceability, and consistency.

E. Security and Trust Assumptions

It is presumed that the model is based on limited-trust approach. The participants' identities can be authenticated but there is no full trust. All the communications between users follow specific communication protocols and rules.

By removing any central point of interaction, it also means that all known attack vectors which target centralized server are gone along with it and aligns security approach with architecture [7], [9], [11].

F. Research Scope and Limitations

This research does not extend to large scale teams because it would not be feasible to use decentralization at scale. There will be no big benchmark tests and advanced trust mechanism studies.

This research aims to explore possible architectural approach, and workflow feasibility which serve as basis for further development in terms of scalability and performance improvement.

IV. Implementation

This section describes the implementation of the decentralized collaborative code editor that has been suggested through a practical implementation. This section focuses on how the components interact with each other, how the workflow takes place and what happens during peer-to-peer synchronisation. Discussion about performance evaluation and behavioural analysis is excluded from this context.

A. System Architecture Overview

Fig. 1 shows the overall architecture of the system. The system is designed to be decentralized whereby every participant acts as a peer node. Every node contains a local code editor, local file storage as well as a distributed version control repository. There is no need for any centralized repository or coordination server for the system [6], [8], [11].

Every peer acts independently and can support any local development task. Communication between different peers is done using peer-to-peer communication channels through network communication protocols. This makes it possible to exchange updates of the repository directly without relying on any external infrastructure [2], [9].

Fig. 2 illustrates the offline-first workflow where editing, staging, and committing happen locally while synchronization happens across peers automatically. Fig. 3 provides a more detailed architectural view of the system.

B. Local Development and Version Control Integration

Each developer works directly from a local editor that is connected to the local file system. Changes in files are recorded locally through the version control interface. The commit process is done in the local distributed version control system without the need for any internet connection [6].

As shown in Fig. 3, the editor and the version control tools are present within each peer node. This allows coding and version control to be performed locally without additional coordination. Data such as commit history, file status, and metadata are stored locally on each node. Thus development can continue even without network connectivity.

C. Automated Commit and Synchronization Triggering

A layer for automation is added between the editor and the version control repository, as depicted in Fig. 2. The automation layer monitors file changes and user actions to trigger commit operations automatically. The idea is to simplify the development process and reduce manual effort.

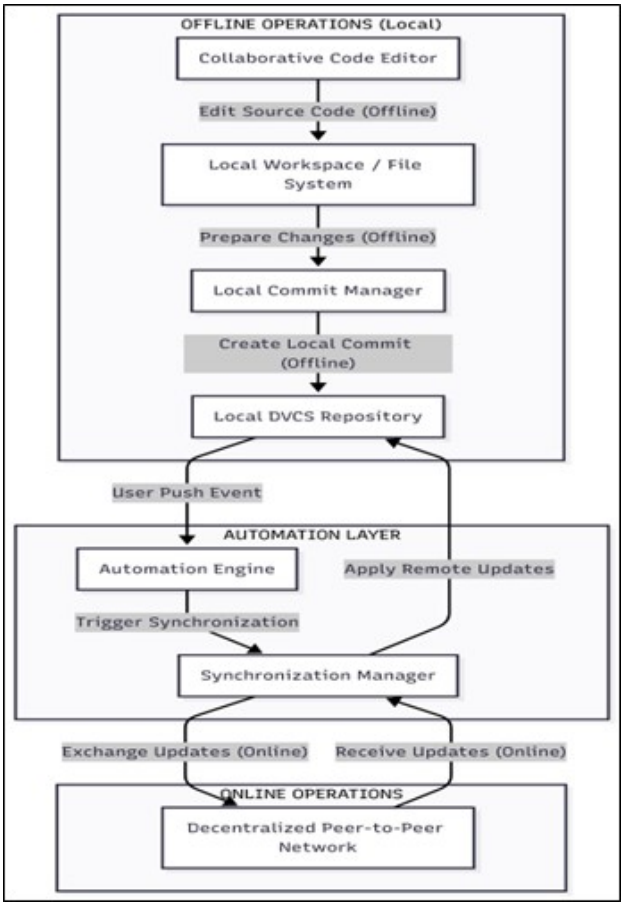

Fig. 2. Offline-first development workflow with automated commit and peer-to-peer synchronization.

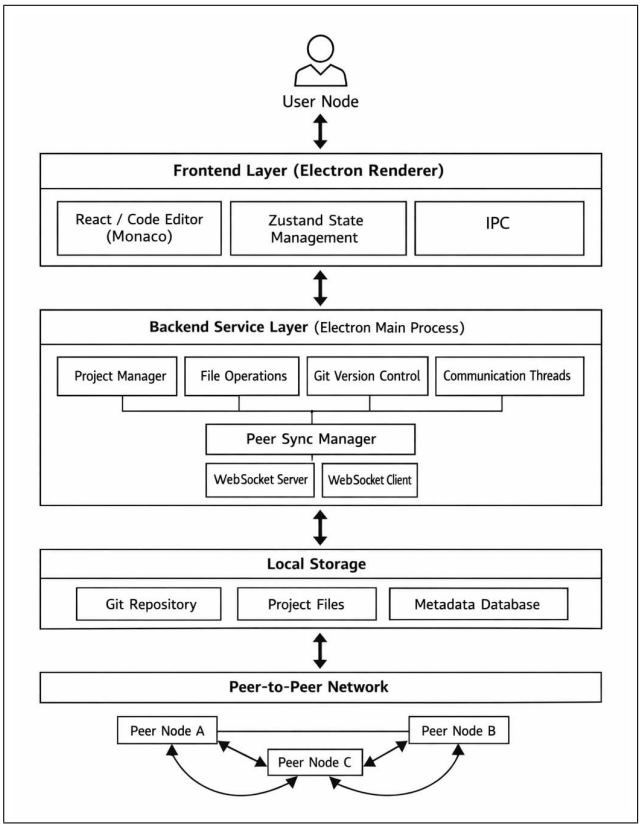

Fig. 3. Detailed system architecture showing frontend components, backend services, local storage, and peer-to-peer communication layers.

The synchronization manager shown in Fig. 3 is responsible for updating the repository in a distributed manner. It prepares commit data, sends updates to other connected peers, and processes incoming updates in the local repository. All these operations are handled locally at each peer.

D. Peer Discovery and Network Management

The peer discovery process, shown in Fig. 2, allows collaborators to discover each other when a new node joins the network. After discovery, peers exchange identity information and repository data to establish direct communication.

Each node maintains its own list of connected peers locally. This list is updated dynamically as peers join or leave the network. Since there is no centralized control, the system supports dynamic participation [2].

E. Peer-to-Peer Repository Synchronization

Synchronization occurs directly between peers through the exchange of repository updates, as illustrated in Fig. 2. Commit information is transferred and merged into the local distributed version control repository [6], [11].

The synchronization mechanism supports two-way communication. Nodes can both send and receive updates, allowing repository states to gradually become consistent over time. Each node maintains its own history independently.

F. Fault Tolerance at the Implementation Level

All editing and version control operations are performed locally before any network communication takes place. If network connectivity is unavailable during synchronization, development continues without interruption. Pending updates are stored locally and synchronized once connectivity is restored.

This behaviour is a direct result of the decentralized and offline-first design [2], [9].

V. Results, Analysis, and Discussion

Observations on the system were made as part of collaboration sessions as the system was being developed and implemented. These observations were made during the editing of code, exchanges of updates, and varied networking situations. It was evident that development could take place independently of having access to a central repository.

Every developer used his/her own repository when making edits, but would then share his/her edits as soon as network access was available. Most of the development took place on the local machine; however, this process did not hinder collaboration.

It is apparent from Fig. 2 that exchanges of information were conducted between the peers without using a central

server at all. It is apparent that collaborative editing works in combination with decentralized version control [3], [5]. The processes of commit and synchronization were carried out automatically, which made it possible to avoid using Git commands manually. This way, developers could devote more time to coding, while the process of synchronization went on silently in the background.

Distributed repository ownership was also an important factor in determining the method of collaboration. Instead of depending on one central repository for version history, each node kept a local version and updated others as required. Over several cycles of synchronization, the repositories became increasingly coherent, much like distributed version control systems [6].

In terms of storage, each collaborator used a separate copy of the repository. This eliminated the requirement for centralized storage, and minimized the impact of individual nodes failing [7], [9].

There were certain practical benefits found in this process. Development proceeded despite unstable connection. It did not require constant internet connection. There was automation involved and developers could have complete control of the repository. However, certain problems also appeared. Manual peer verification had to be done since there was no central identity management system set up. It will be rather difficult to add new peers into the system [7]. The more peers involved in the development process, the more synchronization traffic is generated [6], [8]. It leads to potential delays.

VI. Conclusion

These results confirm that coding cooperation and distribution using version control does not need centralized architecture. In this case, the developer performs all the work in his/her own local repository, while at the same time updates between users can be performed independent from one another. The offline function allows continuing coding in spite of connectivity problems, whereas automatic versioning ensures minimal effort involved. Although some problems related to scalability and trust management exist, this scheme provides feasible answers to such questions.

References

[1] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," in *Concurrency: the Works of Leslie Lamport*, 2019, pp. 179–196.

[2] E. A. Brewer, "Towards robust distributed systems," *PODC*, vol. 7, no. 10.1145, 2000.

[3] M. Shapiro *et al.*, "Conflict-free replicated data types," in *Symposium on Self-Stabilizing Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011.

[4] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, 1989.

[5] N. Preguiça *et al.*, "A commutative replicated data type for cooperative editing," in *2009 29th IEEE International Conference on Distributed Computing Systems*. IEEE, 2009.

[6] A. van der Hoek, D. Redmiles, J. Herbsleb, and E. H. Chi, "Designing for distributed software development," in *Proc. Int. Conf. Software Engineering (ICSE)*, 2004, pp. 356–365.

[7] Y. Saito and M. Shapiro, "Optimistic replication," *ACM Computing Surveys (CSUR)*, vol. 37, no. 1, pp. 42–81, 2005.

[8] A. D. Birrell and B. J. Nelson, "Implementing remote procedure calls," *ACM Transactions on Computer Systems (TOCS)*, vol. 2, no. 1, pp. 39–59, 1984.

[9] B. Nasrulin and J. Pouwelse, "Decentralized Collaborative Version Control," in *Proceedings of the 2nd International Workshop on Distributed Infrastructure for Common Good*, 2021.

[10] M. R. Haque *et al.*, "An integrated blockchain and IPFS-based solution for secure and efficient source code repository hosting using middleman approach," *PLOS One*, vol. 20, no. 9, p. e0331131, 2025.

[11] I. David, E. Syriani, and C. Masson, "Extensible Conflict-Free Replicated Datatypes for Real-time Collaborative Software Engineering," in *2022 17th Conference on Computer Science and Intelligence Systems (FedCSIS)*, Sofia, Bulgaria, 2022, pp. 849–853, doi: 10.15439/2022F99.

[12] H. Pathak, H. Ritik, R. Pawar, and Y. Pingle, "CodeFlow: Real-Time Collaborative Code Editor," in *2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS)*, Chikkaballapur, India, 2024, pp. 1–5, doi: 10.1109/ICKECS61492.2024.10617308.

[13] E. Karayel and E. Gonzàlez, "Strong eventual consistency of the collaborative editing framework WOOT," *Distributed Computing*, vol. 35, no. 2, pp. 145–164, Apr. 2022, doi: 10.1007/s00446-021-00414-6.